

SQL MINI PROJECT- RETAIL STORE

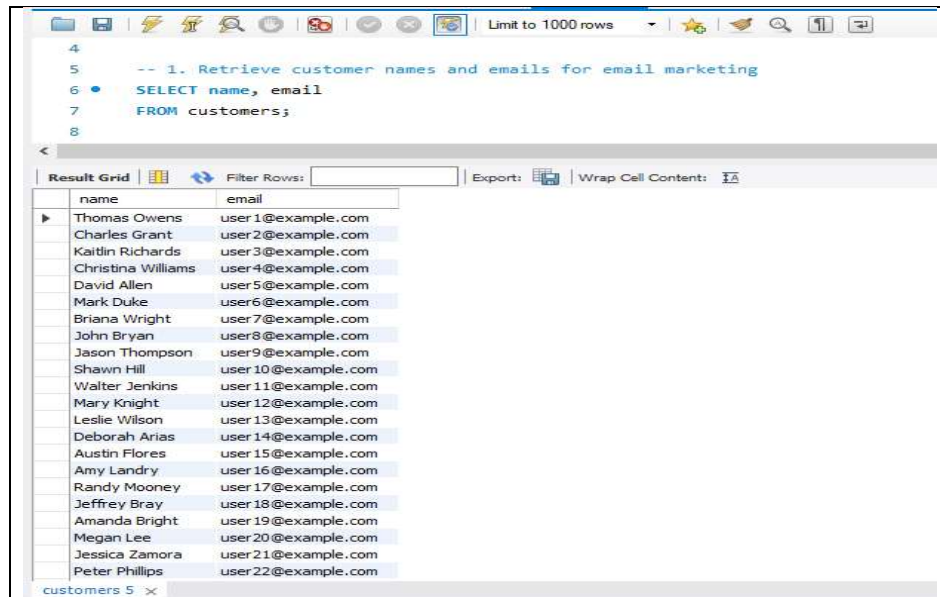
YUKTI MAHOBIA

LEVEL 1: BASICS

1. SOLUTION QUERY:

```
SELECT name, email
FROM customers;
```

OUTPUT:



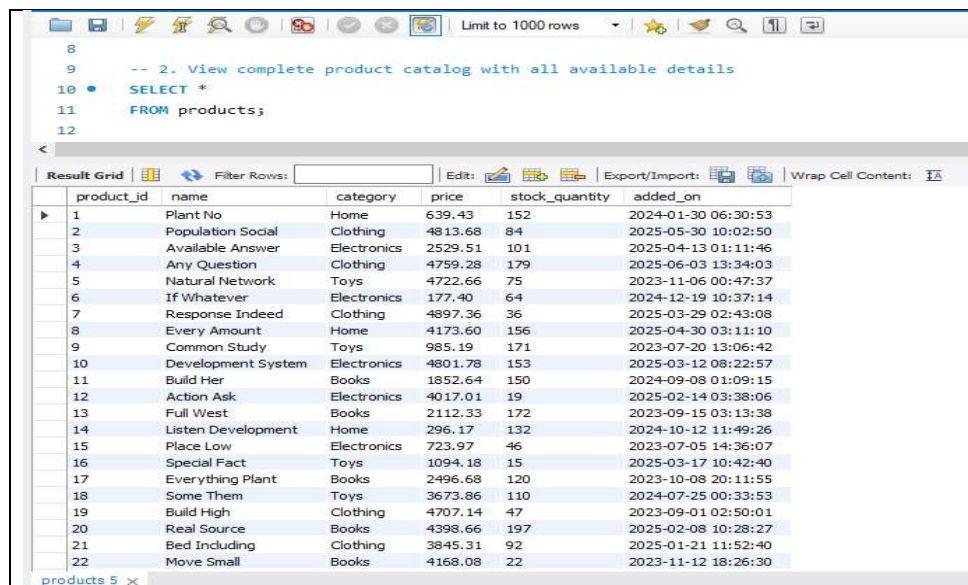
```
-- 1. Retrieve customer names and emails for email marketing
SELECT name, email
FROM customers;
```

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com
Jeffrey Bray	user18@example.com
Amanda Bright	user19@example.com
Megan Lee	user20@example.com
Jessica Zamora	user21@example.com
Peter Phillips	user22@example.com

2. SOLUTION QUERY:

```
SELECT *
FROM products;
```

OUTPUT:



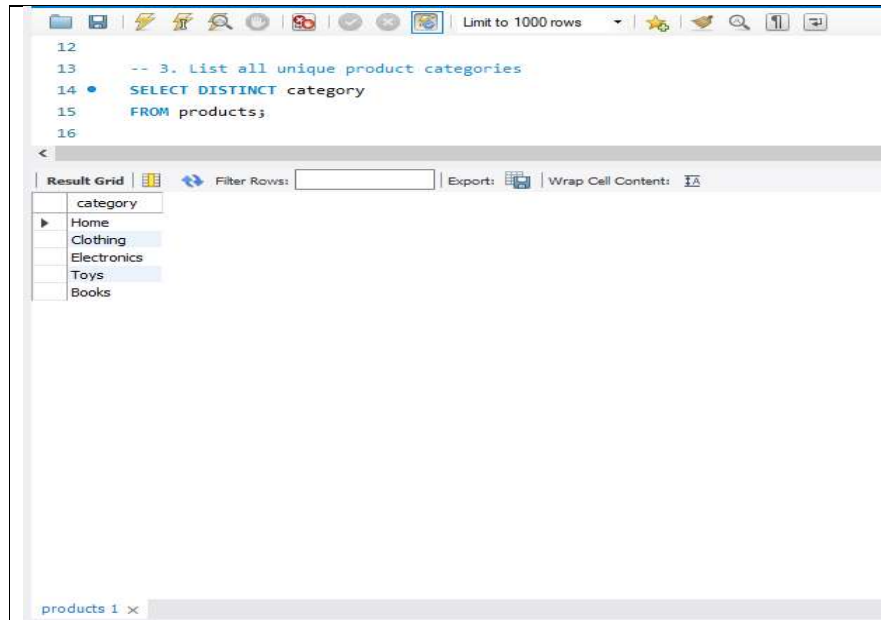
```
-- 2. View complete product catalog with all available details
SELECT *
FROM products;
```

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	IF Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

3. SOLUTION QUERY:

```
SELECT DISTINCT category
FROM products;
```

OUTPUT:



The screenshot shows a database query editor with the following SQL query:

```
-- 3. List all unique product categories
SELECT DISTINCT category
FROM products;
```

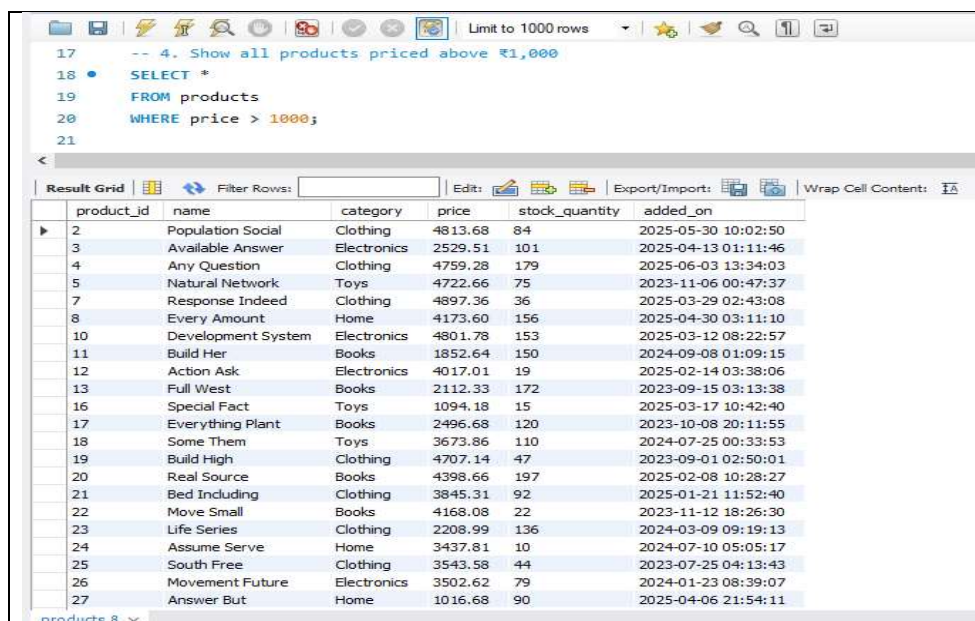
The output is displayed in a table with the following categories:

category
Home
Clothing
Electronics
Toys
Books

4. SOLUTION QUERY:

```
SELECT *
FROM products
WHERE price > 1000;
```

OUTPUT:



The screenshot shows a database query editor with the following SQL query:

```
-- 4. Show all products priced above ₹1,000
SELECT *
FROM products
WHERE price > 1000;
```

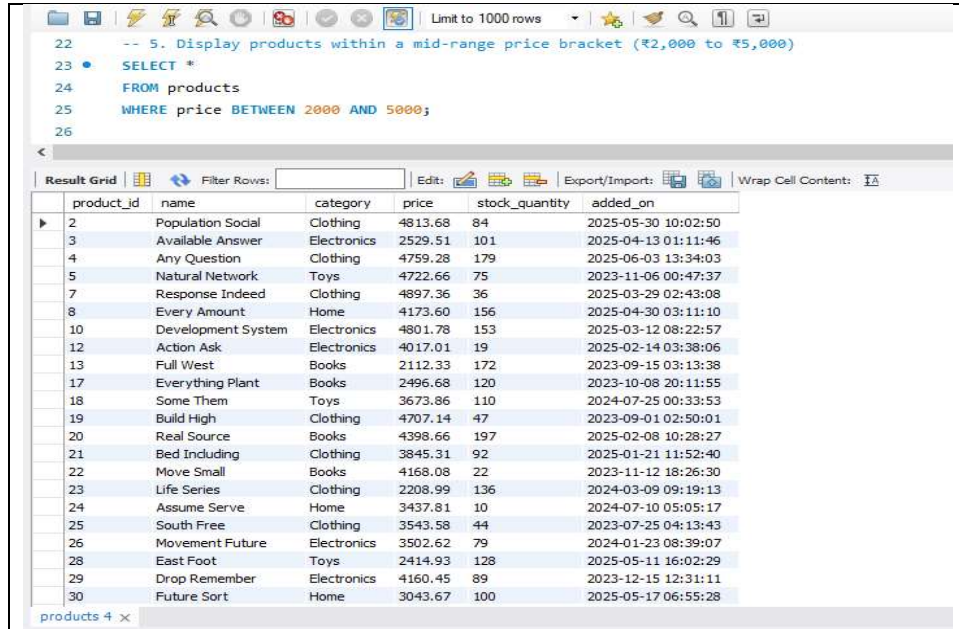
The output is displayed in a table with the following columns: product_id, name, category, price, stock_quantity, and added_on. The table contains 27 rows of data.

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
27	Answer But	Home	1016.68	90	2025-04-06 21:54:11

5. SOLUTION QUERY:

```
SELECT *  
FROM products  
WHERE price BETWEEN 2000 AND 5000;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
-- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)  
SELECT *  
FROM products  
WHERE price BETWEEN 2000 AND 5000;
```

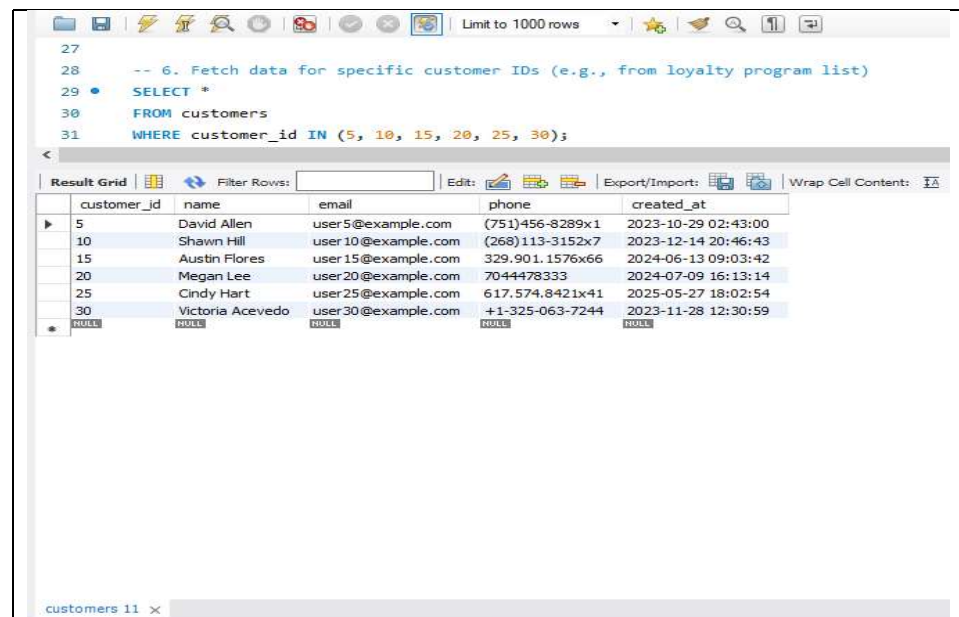
Below the query, the 'Result Grid' displays the results of the query. The table has the following columns: product_id, name, category, price, stock_quantity, and added_on. The results are as follows:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
28	East Foot	Toys	2414.93	128	2025-05-11 16:02:29
29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28

6. SOLUTION QUERY:

```
SELECT *  
FROM customers  
WHERE customer_id IN (5, 10, 15, 20, 25, 30);
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
-- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)  
SELECT *  
FROM customers  
WHERE customer_id IN (5, 10, 15, 20, 25, 30);
```

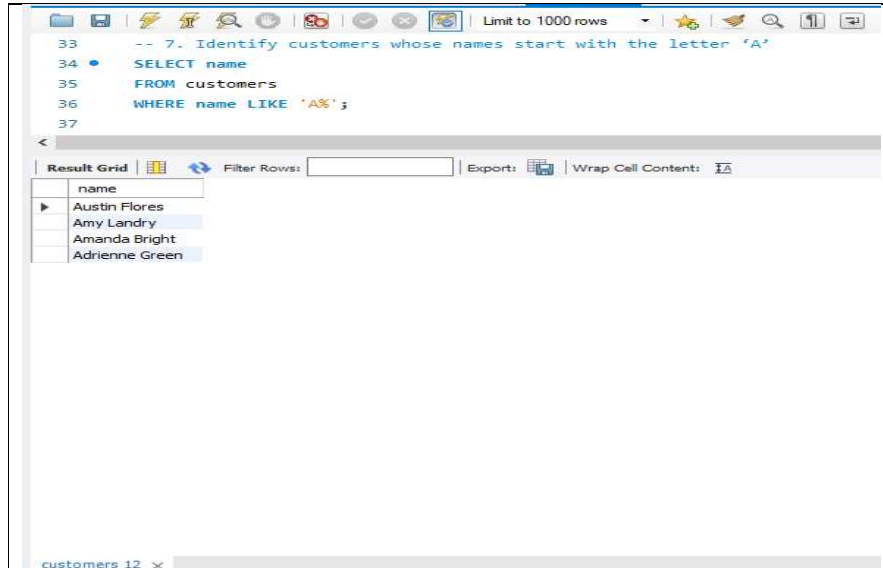
Below the query, the 'Result Grid' displays the results of the query. The table has the following columns: customer_id, name, email, phone, and created_at. The results are as follows:

customer_id	name	email	phone	created_at
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14
25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54
30	Victoria Acevedo	user30@example.com	+1-325-063-7244	2023-11-28 12:30:59
*	NULL	NULL	NULL	NULL

7. SOLUTION QUERY:

```
SELECT name
FROM customers
WHERE name LIKE 'A%';
```

OUTPUT:



The screenshot shows a database query editor with the following SQL query:

```
-- 7. Identify customers whose names start with the letter 'A'
SELECT name
FROM customers
WHERE name LIKE 'A%';
```

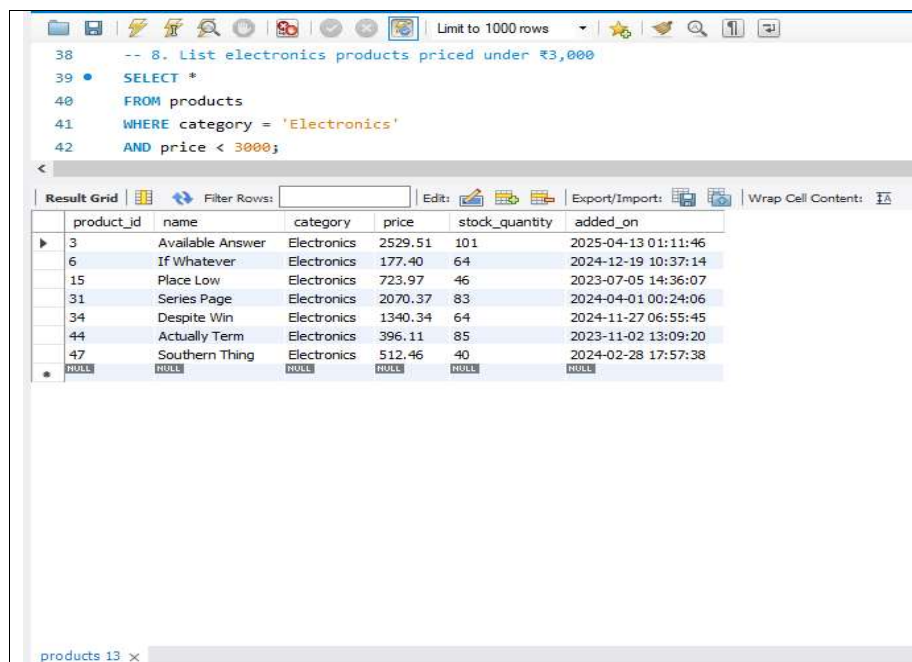
The results are displayed in a table with the following data:

name
Austin Flores
Amy Landry
Amanda Bright
Adrienne Green

8. SOLUTION QUERY:

```
SELECT *
FROM products
WHERE category = 'Electronics'
AND price < 3000;
```

OUTPUT:



The screenshot shows a database query editor with the following SQL query:

```
-- 8. List electronics products priced under ₹3,000
SELECT *
FROM products
WHERE category = 'Electronics'
AND price < 3000;
```

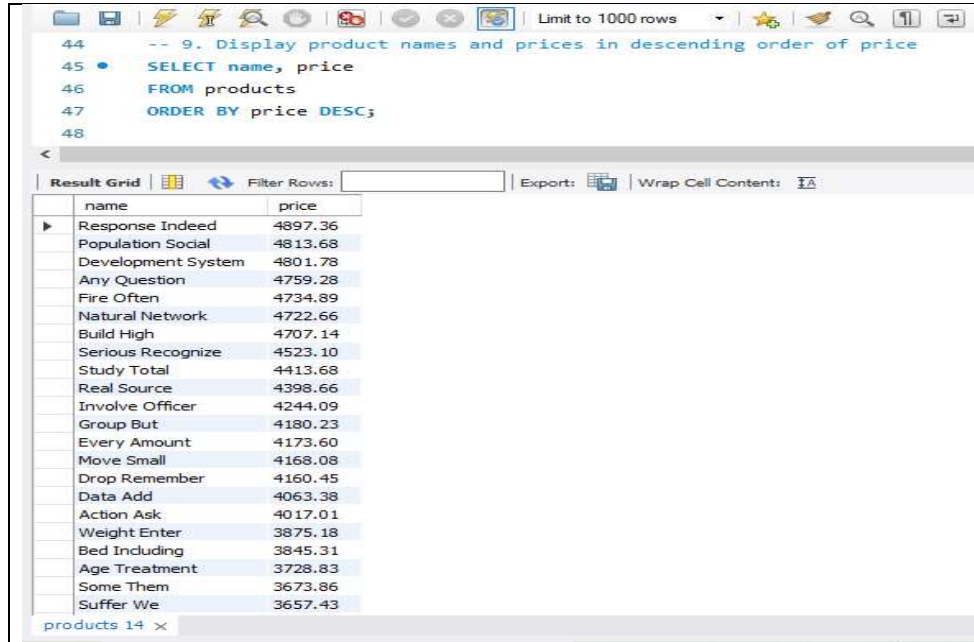
The results are displayed in a table with the following data:

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
NULL	NULL	NULL	NULL	NULL	NULL

9. SOLUTION QUERY:

```
SELECT name, price
FROM products
ORDER BY price DESC;
```

OUTPUT:



Limit to 1000 rows

```
-- 9. Display product names and prices in descending order of price
45 • SELECT name, price
46 FROM products
47 ORDER BY price DESC;
48
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

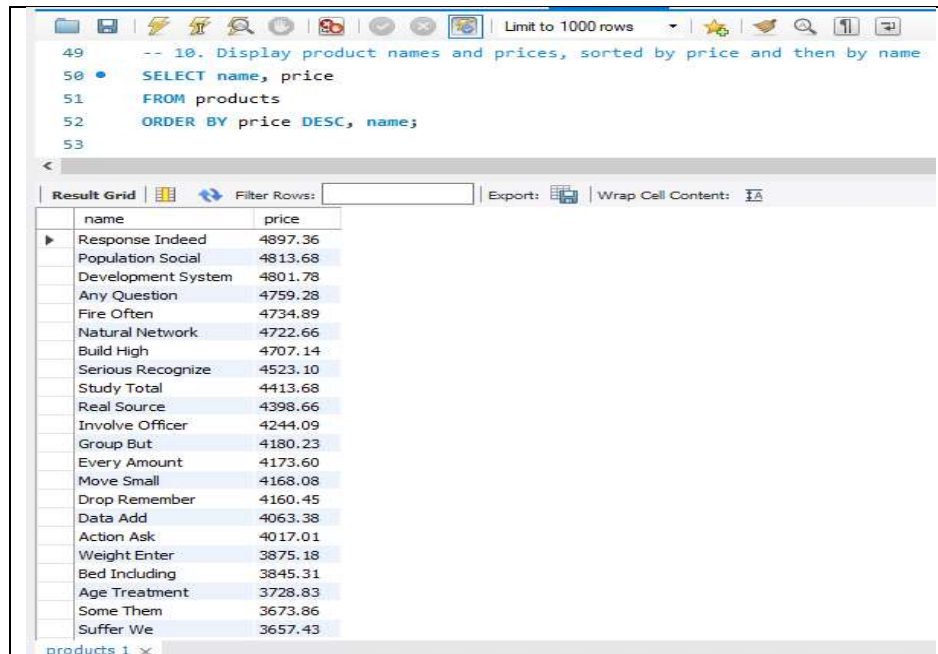
name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18
Bed Including	3845.31
Age Treatment	3728.83
Some Them	3673.86
Suffer We	3657.43

products 14 x

10. SOLUTION QUERY:

```
SELECT name, price
FROM products
ORDER BY price DESC, name;
```

OUTPUT:



Limit to 1000 rows

```
-- 10. Display product names and prices, sorted by price and then by name
50 • SELECT name, price
51 FROM products
52 ORDER BY price DESC, name;
53
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18
Bed Including	3845.31
Age Treatment	3728.83
Some Them	3673.86
Suffer We	3657.43

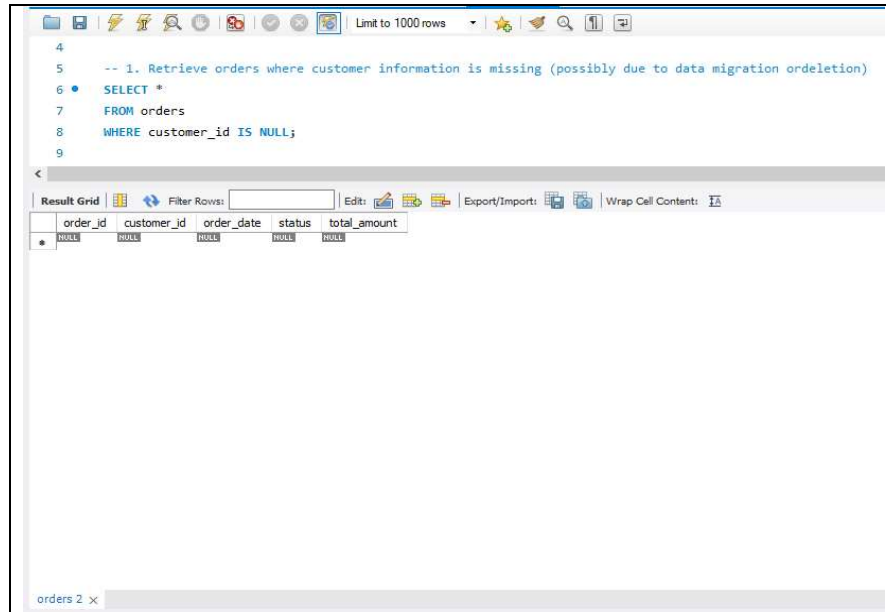
products 1 x

LEVEL 2: FILTERING AND FORMATTING

1. SOLUTION QUERY:

```
SELECT *  
FROM orders  
WHERE customer_id IS NULL;
```

OUTPUT:



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
4  
5 -- 1. Retrieve orders where customer information is missing (possibly due to data migration ordeletion)  
6 • SELECT *  
7 FROM orders  
8 WHERE customer_id IS NULL;  
9
```

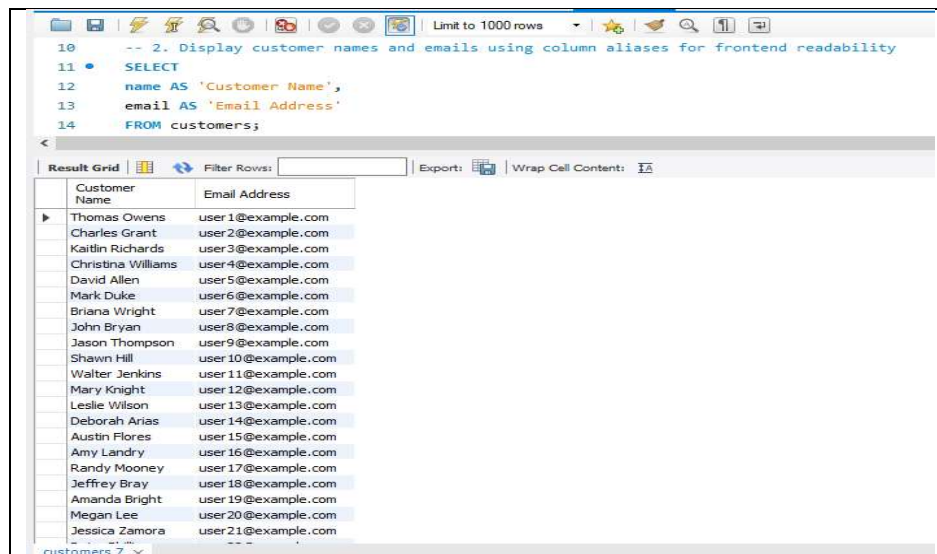
The result grid shows the output of the query. The columns are: order_id, customer_id, order_date, status, and total_amount. The first row shows all NULL values.

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

2. SOLUTION QUERY:

```
SELECT  
name AS 'Customer Name',  
email AS 'Email Address'  
FROM customers;
```

OUTPUT:



The screenshot shows a SQL IDE window with a query and its output. The query editor contains the following SQL code:

```
10 -- 2. Display customer names and emails using column aliases for frontend readability  
11 • SELECT  
12 name AS 'Customer Name',  
13 email AS 'Email Address'  
14 FROM customers;
```

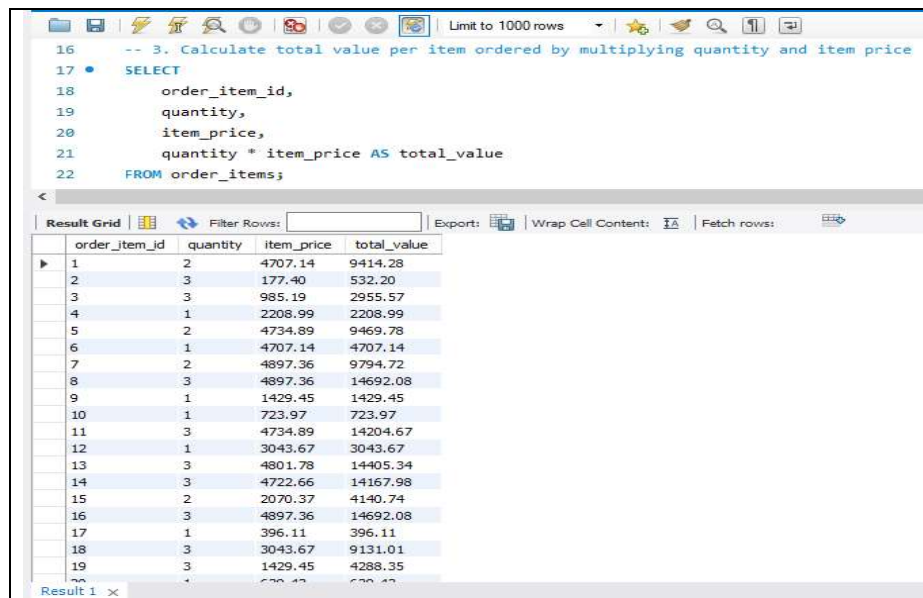
The result grid shows the output of the query. The columns are: Customer Name and Email Address. The output lists 21 customers with their names and email addresses.

Customer Name	Email Address
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com
Jeffrey Bray	user18@example.com
Amanda Bright	user19@example.com
Megan Lee	user20@example.com
Jessica Zamora	user21@example.com

3. SOLUTION QUERY:

```
SELECT
    order_item_id,
    quantity,
    item_price,
    quantity * item_price AS total_value
FROM order_items;
```

OUTPUT:

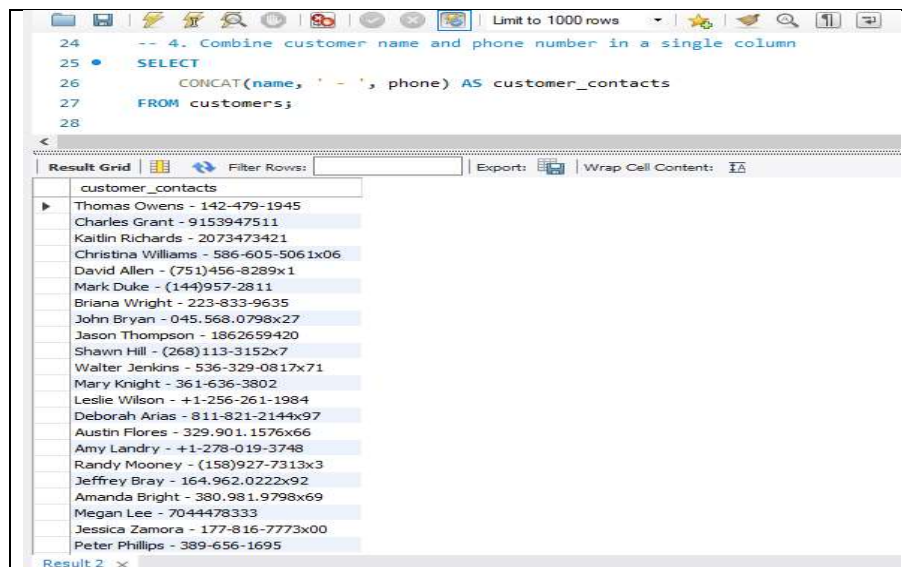


order_item_id	quantity	item_price	total_value
1	2	4707.14	9414.28
2	3	177.40	532.20
3	3	985.19	2955.57
4	1	2208.99	2208.99
5	2	4734.89	9469.78
6	1	4707.14	4707.14
7	2	4897.36	9794.72
8	3	4897.36	14692.08
9	1	1429.45	1429.45
10	1	723.97	723.97
11	3	4734.89	14204.67
12	1	3043.67	3043.67
13	3	4801.78	14405.34
14	3	4722.66	14167.98
15	2	2070.37	4140.74
16	3	4897.36	14692.08
17	1	396.11	396.11
18	3	3043.67	9131.01
19	3	1429.45	4288.35
20	1	620.42	620.42

4. SOLUTION QUERY:

```
SELECT
    CONCAT(name, ' - ', phone) AS customer_contacts
FROM customers;
```

OUTPUT:



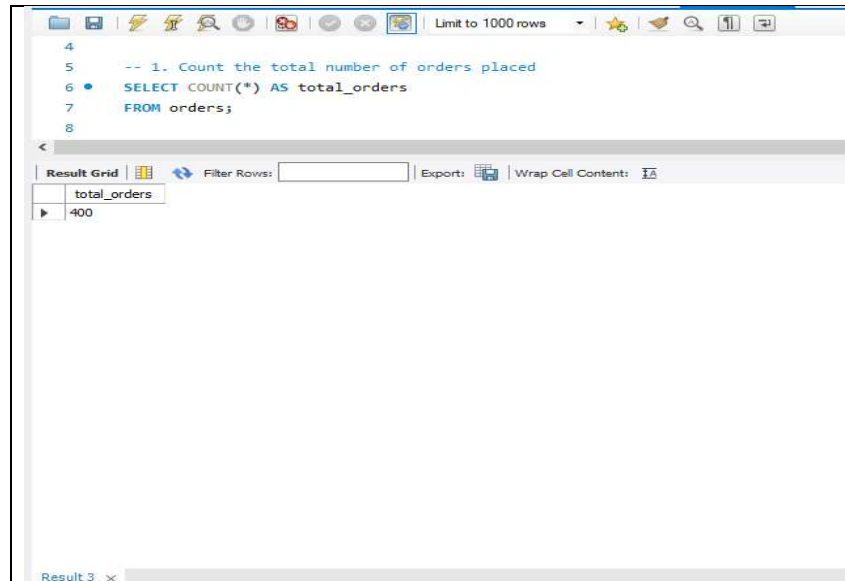
customer_contacts
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7
Walter Jenkins - 536-329-0817x71
Mary Knight - 361-636-3802
Leslie Wilson - +1-256-261-1984
Deborah Arias - 811-821-2144x97
Austin Flores - 329.901.1576x66
Amy Landry - +1-278-019-3748
Randy Mooney - (158)927-7313x3
Jeffrey Bray - 164.962.0222x92
Amanda Bright - 380.981.9798x69
Megan Lee - 7044478333
Jessica Zamora - 177-816-7773x00
Peter Phillips - 389-656-1695

LEVEL 3: AGGREGATIONS

1. SOLUTION QUERY:

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query text is as follows:

```
4  
5 -- 1. Count the total number of orders placed  
6 • SELECT COUNT(*) AS total_orders  
7 FROM orders;  
8
```

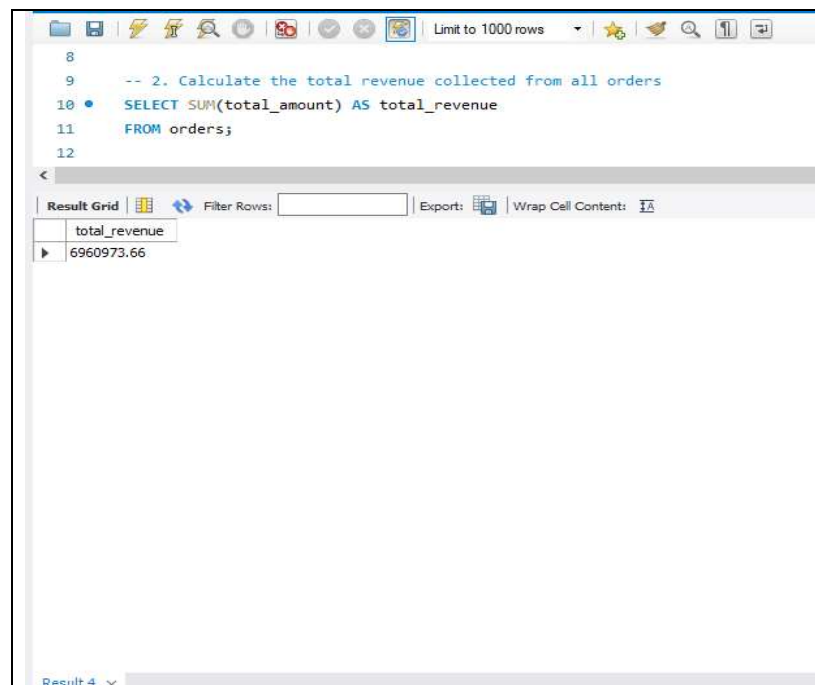
Below the query editor, the 'Result Grid' is displayed. It shows a single column named 'total_orders' with a value of 400. The interface includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The status bar at the bottom indicates 'Result 3'.

total_orders
400

2. SOLUTION QUERY:

```
SELECT SUM(total_amount) AS total_revenue  
FROM orders;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query text is as follows:

```
8  
9 -- 2. Calculate the total revenue collected from all orders  
10 • SELECT SUM(total_amount) AS total_revenue  
11 FROM orders;  
12
```

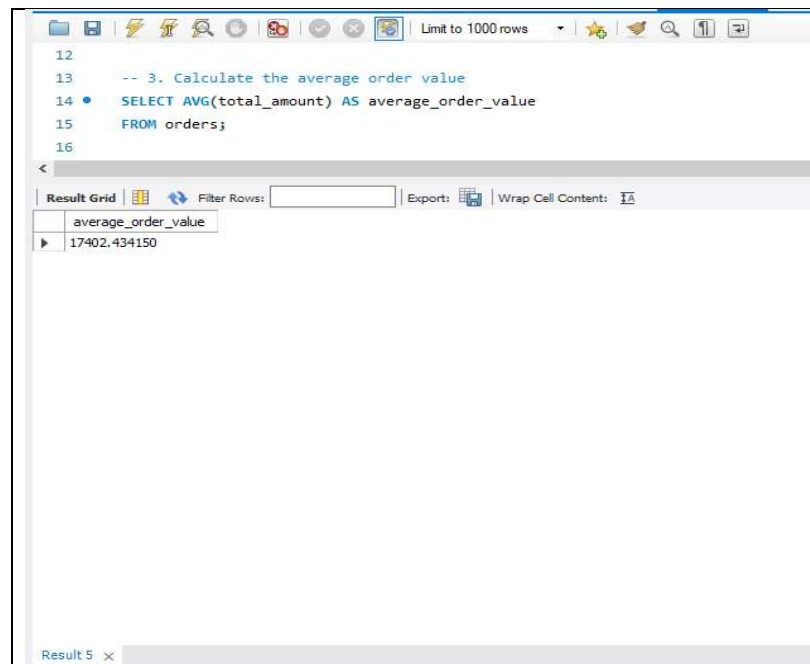
Below the query editor, the 'Result Grid' is displayed. It shows a single column named 'total_revenue' with a value of 6960973.66. The interface includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The status bar at the bottom indicates 'Result 4'.

total_revenue
6960973.66

3. SOLUTION QUERY:

```
SELECT AVG(total_amount) AS average_order_value  
FROM orders;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
12  
13 -- 3. Calculate the average order value  
14 • SELECT AVG(total_amount) AS average_order_value  
15 FROM orders;  
16
```

Below the query editor, the 'Result Grid' is displayed. It has a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result is a single row with the column 'average_order_value' and the value '17402.434150'.

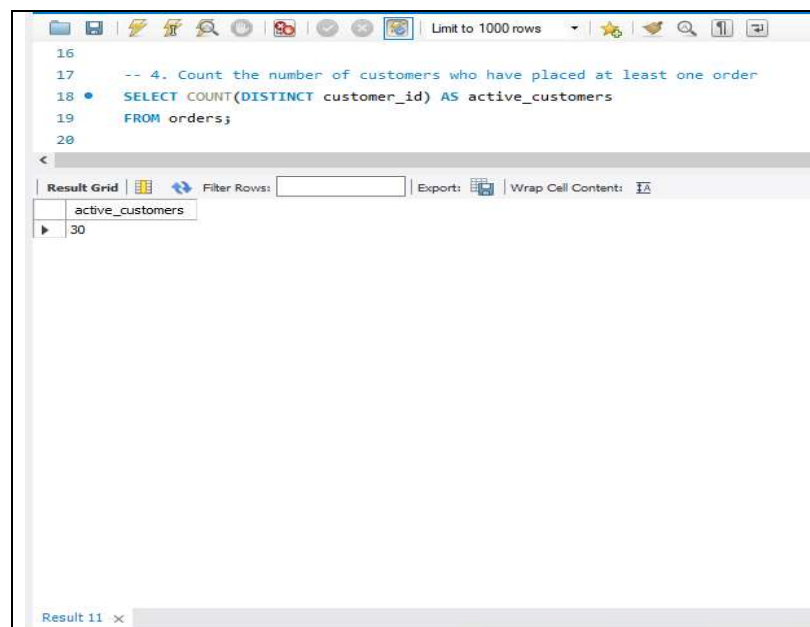
average_order_value
17402.434150

The bottom of the window shows 'Result 5' with a close button.

4. SOLUTION QUERY:

```
SELECT COUNT(DISTINCT customer_id) AS active_customers  
FROM orders;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
16  
17 -- 4. Count the number of customers who have placed at least one order  
18 • SELECT COUNT(DISTINCT customer_id) AS active_customers  
19 FROM orders;  
20
```

Below the query editor, the 'Result Grid' is displayed. It has a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result is a single row with the column 'active_customers' and the value '30'.

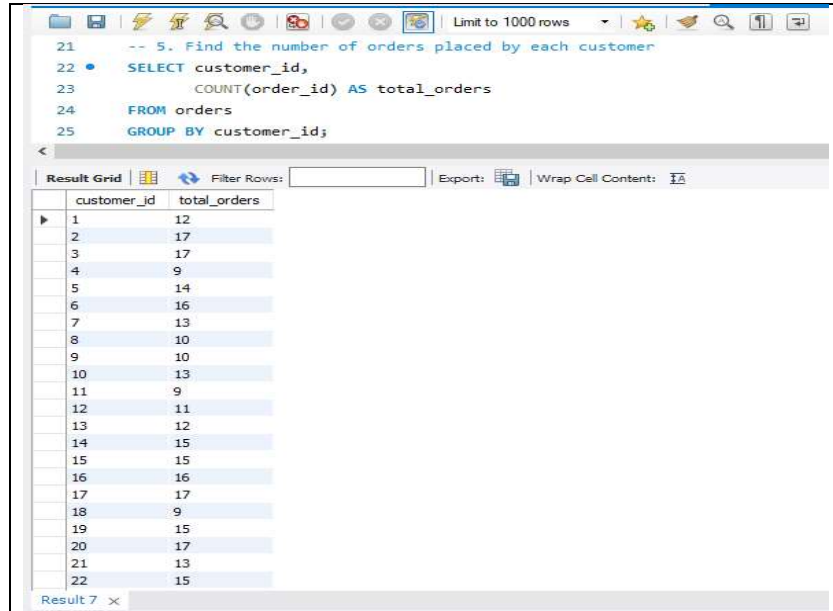
active_customers
30

The bottom of the window shows 'Result 11' with a close button.

5. SOLUTION QUERY:

```
SELECT customer_id,  
       COUNT(order_id) AS total_orders  
FROM orders  
GROUP BY customer_id;
```

OUTPUT:



The screenshot shows a SQL query editor with the following query:

```
-- 5. Find the number of orders placed by each customer  
SELECT customer_id,  
       COUNT(order_id) AS total_orders  
FROM orders  
GROUP BY customer_id;
```

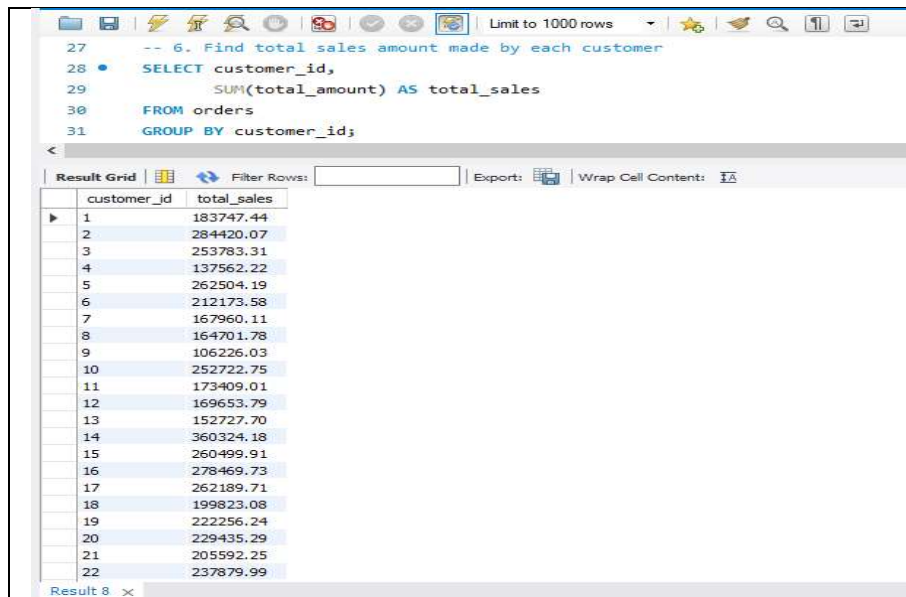
The result grid displays the following data:

customer_id	total_orders
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11
13	12
14	15
15	15
16	16
17	17
18	9
19	15
20	17
21	13
22	15

6. SOLUTION QUERY:

```
SELECT customer_id,  
       SUM(total_amount) AS total_sales  
FROM orders  
GROUP BY customer_id;
```

OUTPUT:



The screenshot shows a SQL query editor with the following query:

```
-- 6. Find total sales amount made by each customer  
SELECT customer_id,  
       SUM(total_amount) AS total_sales  
FROM orders  
GROUP BY customer_id;
```

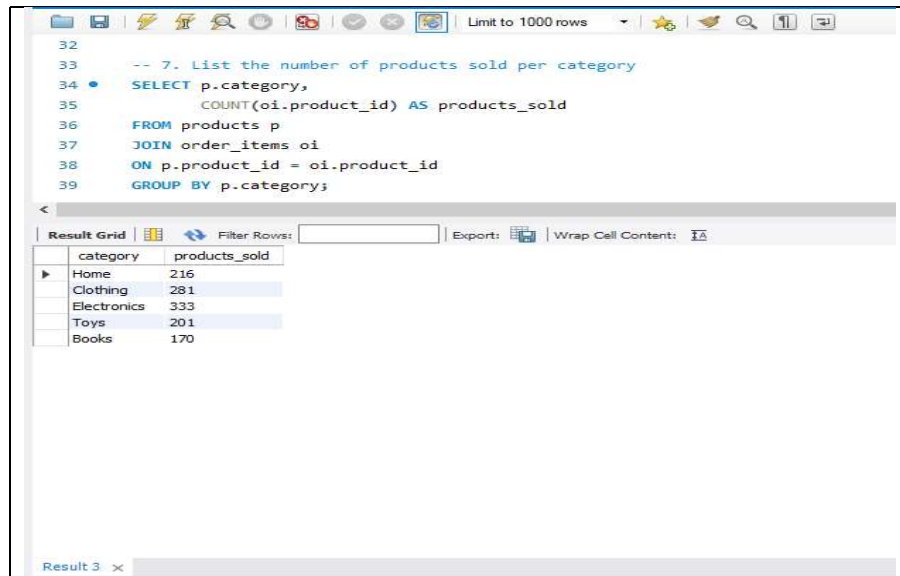
The result grid displays the following data:

customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79
13	152727.70
14	360324.18
15	260499.91
16	278469.73
17	262189.71
18	199823.08
19	222256.24
20	229435.29
21	205592.25
22	237879.99

7. SOLUTION QUERY:

```
SELECT p.category,  
       COUNT(oi.product_id) AS products_sold  
FROM products p  
JOIN order_items oi  
ON p.product_id = oi.product_id  
GROUP BY p.category;
```

OUTPUT:



The screenshot shows a SQL IDE window with a query editor and a results grid. The query editor contains the following SQL code:

```
32  
33 -- 7. List the number of products sold per category  
34 • SELECT p.category,  
35       COUNT(oi.product_id) AS products_sold  
36 FROM products p  
37 JOIN order_items oi  
38 ON p.product_id = oi.product_id  
39 GROUP BY p.category;
```

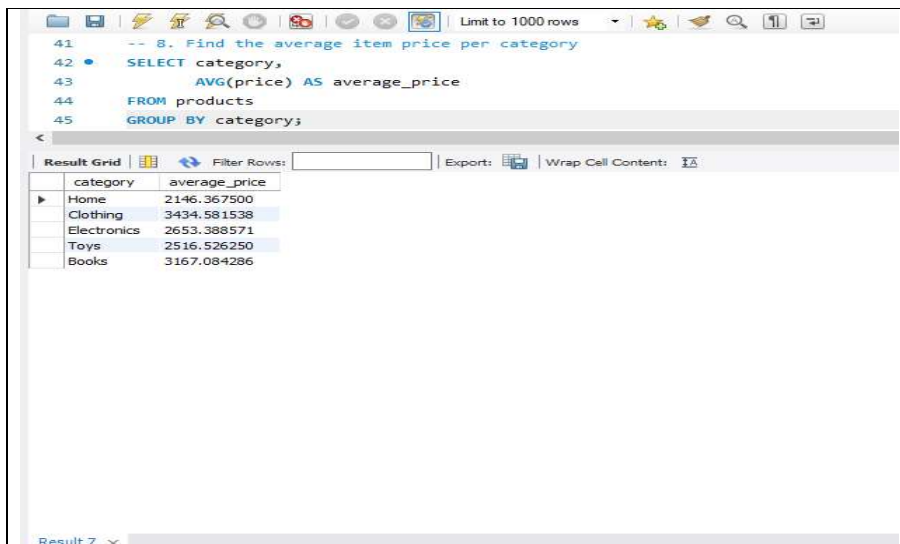
The results grid displays the output of the query:

category	products_sold
Home	216
Clothing	281
Electronics	333
Toys	201
Books	170

8. SOLUTION QUERY:

```
SELECT category,  
       AVG(price) AS average_price  
FROM products  
GROUP BY category;
```

OUTPUT:



The screenshot shows a SQL IDE window with a query editor and a results grid. The query editor contains the following SQL code:

```
41 -- 8. Find the average item price per category  
42 • SELECT category,  
43       AVG(price) AS average_price  
44 FROM products  
45 GROUP BY category;
```

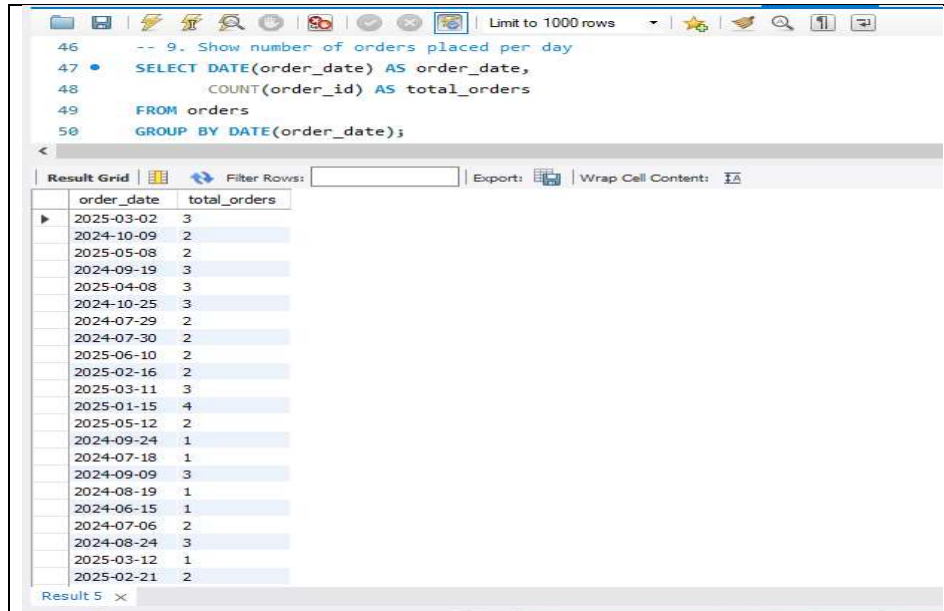
The results grid displays the output of the query:

category	average_price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

9. SOLUTION QUERY:

```
SELECT DATE(order_date) AS order_date,  
       COUNT(order_id) AS total_orders  
FROM orders  
GROUP BY DATE(order_date);
```

OUTPUT:



The screenshot shows a SQL query editor with the following query:

```
-- 9. Show number of orders placed per day  
SELECT DATE(order_date) AS order_date,  
       COUNT(order_id) AS total_orders  
FROM orders  
GROUP BY DATE(order_date);
```

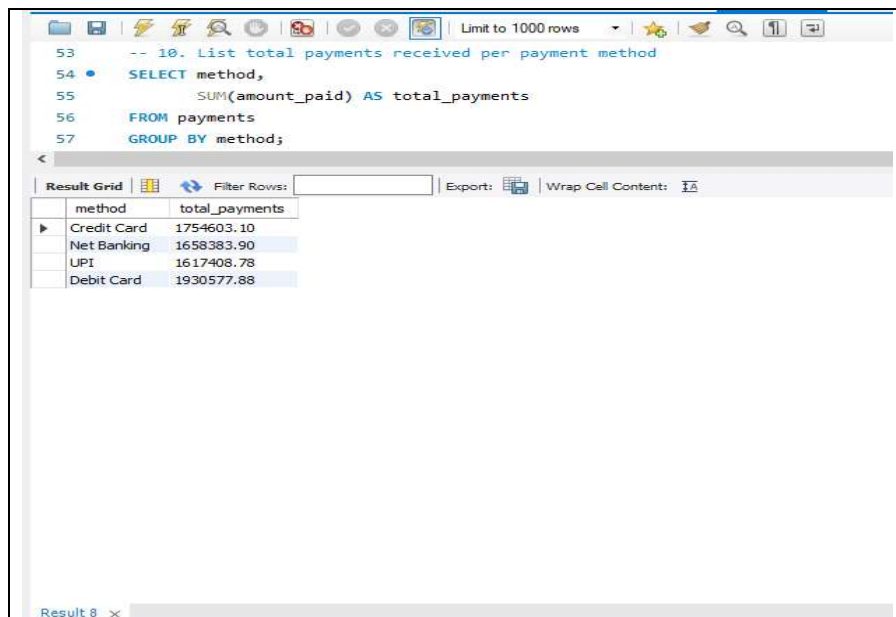
The results are displayed in a grid with the following data:

order_date	total_orders
2025-03-02	3
2024-10-09	2
2025-05-08	2
2024-09-19	3
2025-04-08	3
2024-10-25	3
2024-07-29	2
2024-07-30	2
2025-06-10	2
2025-02-16	2
2025-03-11	3
2025-01-15	4
2025-05-12	2
2024-09-24	1
2024-07-18	1
2024-09-09	3
2024-08-19	1
2024-06-15	1
2024-07-06	2
2024-08-24	3
2025-03-12	1
2025-02-21	2

10. SOLUTION QUERY:

```
SELECT method,  
       SUM(amount_paid) AS total_payments  
FROM payments  
GROUP BY method;
```

OUTPUT:



The screenshot shows a SQL query editor with the following query:

```
-- 10. List total payments received per payment method  
SELECT method,  
       SUM(amount_paid) AS total_payments  
FROM payments  
GROUP BY method;
```

The results are displayed in a grid with the following data:

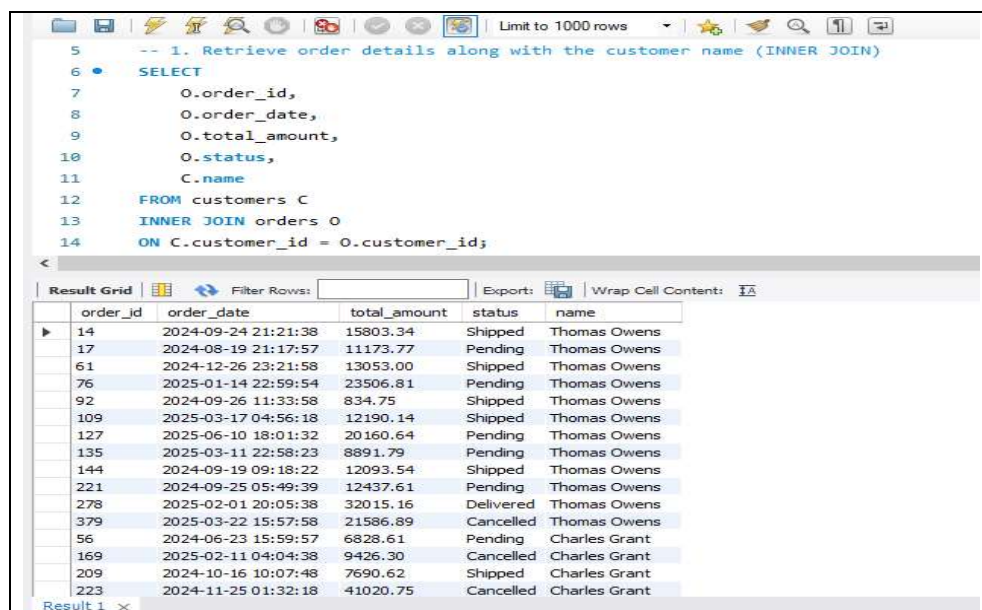
method	total_payments
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

LEVEL 4: MULTI-TABLE QUERIES (JOIN)

1. SOLUTION QUERY:

```
SELECT
    O.order_id,
    O.order_date,
    O.total_amount,
    O.status,
    C.name
FROM customers C
INNER JOIN orders O
ON C.customer_id = O.customer_id;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
-- 1. Retrieve order details along with the customer name (INNER JOIN)
SELECT
    O.order_id,
    O.order_date,
    O.total_amount,
    O.status,
    C.name
FROM customers C
INNER JOIN orders O
ON C.customer_id = O.customer_id;
```

Below the query, the results are displayed in a table with the following columns: order_id, order_date, total_amount, status, and name. The results are filtered to show 1000 rows.

order_id	order_date	total_amount	status	name
14	2024-09-24 21:21:38	15803.34	Shipped	Thomas Owens
17	2024-08-19 21:17:57	11173.77	Pending	Thomas Owens
61	2024-12-26 23:21:58	13053.00	Shipped	Thomas Owens
76	2025-01-14 22:59:54	23506.81	Pending	Thomas Owens
92	2024-09-26 11:33:58	834.75	Shipped	Thomas Owens
109	2025-03-17 04:56:18	12190.14	Shipped	Thomas Owens
127	2025-06-10 18:01:32	20160.64	Pending	Thomas Owens
135	2025-03-11 22:58:23	8891.79	Pending	Thomas Owens
144	2024-09-19 09:18:22	12093.54	Shipped	Thomas Owens
221	2024-09-25 05:49:39	12437.61	Pending	Thomas Owens
278	2025-02-01 20:05:38	32015.16	Delivered	Thomas Owens
379	2025-03-22 15:57:58	21586.89	Cancelled	Thomas Owens
56	2024-06-23 15:59:57	6828.61	Pending	Charles Grant
169	2025-02-11 04:04:38	9426.30	Cancelled	Charles Grant
209	2024-10-16 10:07:48	7690.62	Shipped	Charles Grant
223	2024-11-25 01:32:18	41020.75	Cancelled	Charles Grant

2. SOLUTION QUERY:

```
SELECT
    P.product_id,
    P.name,
    P.category,
    OI.order_id,
    OI.order_item_id
FROM products P
INNER JOIN order_items OI
ON P.product_id = OI.product_id;
```

OUTPUT:

Limit to 1000 rows

```

16  -- 2. Get list of products that have been sold (INNER JOIN with order_items)
17  SELECT
18      P.product_id,
19      P.name,
20      P.category,
21      OI.order_id,
22      OI.order_item_id
23  FROM products P
24  INNER JOIN order_items OI
25  ON P.product_id = OI.product_id;

```

Result Grid

	product_id	name	category	order_id	order_item_id
1	1	Plant No	Home	10	20
	1	Plant No	Home	11	23
	1	Plant No	Home	28	72
	1	Plant No	Home	37	103
	1	Plant No	Home	49	140
	1	Plant No	Home	71	210
	1	Plant No	Home	75	223
	1	Plant No	Home	81	248
	1	Plant No	Home	127	372
	1	Plant No	Home	134	398
	1	Plant No	Home	171	493
	1	Plant No	Home	236	682
	1	Plant No	Home	239	695
	1	Plant No	Home	288	851
	1	Plant No	Home	314	934

Result 2 x

3. SOLUTION QUERY:

```

SELECT
    O.order_id,
    P.method
FROM orders O
INNER JOIN payments P
ON O.order_id = P.order_id;

```

OUTPUT:

Limit to 1000 rows

```

27  -- 3. List all orders with their payment method (INNER JOIN)
28  SELECT
29      O.order_id,
30      P.method
31  FROM orders O
32  INNER JOIN payments P
33  ON O.order_id = P.order_id;
34

```

Result Grid

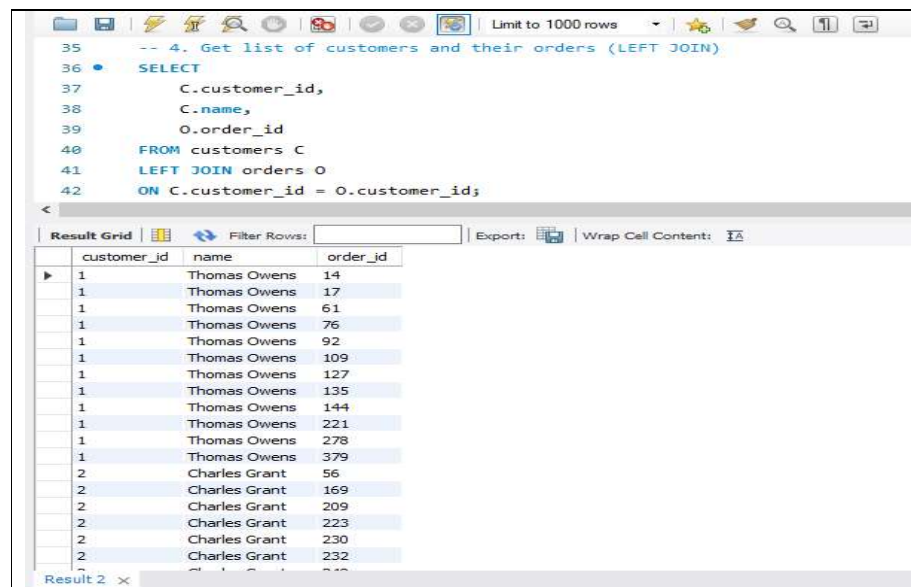
	order_id	method
1	1	Credit Card
2	2	Net Banking
3	3	Credit Card
4	4	UPI
5	5	UPI
6	6	UPI
7	7	UPI
8	8	Net Banking
9	9	Net Banking
10	10	Debit Card
11	11	Credit Card
12	12	Credit Card
13	13	UPI
14	14	UPI
15	15	UPI
16	16	UPI
17	17	UPI
18	18	Net Banking
19	19	Net Banking

Result 1 x

4. SOLUTION QUERY:

```
SELECT
    C.customer_id,
    C.name,
    O.order_id
FROM customers C
LEFT JOIN orders O
ON C.customer_id = O.customer_id;
```

OUTPUT:



The screenshot shows a database query editor with the following SQL query:

```
-- 4. Get list of customers and their orders (LEFT JOIN)
SELECT
    C.customer_id,
    C.name,
    O.order_id
FROM customers C
LEFT JOIN orders O
ON C.customer_id = O.customer_id;
```

The results are displayed in a grid with the following columns: customer_id, name, and order_id. The grid shows 19 rows of data, with 10 rows for customer_id 1 (Thomas Owens) and 9 rows for customer_id 2 (Charles Grant).

customer_id	name	order_id
1	Thomas Owens	14
1	Thomas Owens	17
1	Thomas Owens	61
1	Thomas Owens	76
1	Thomas Owens	92
1	Thomas Owens	109
1	Thomas Owens	127
1	Thomas Owens	135
1	Thomas Owens	144
1	Thomas Owens	221
1	Thomas Owens	278
1	Thomas Owens	379
2	Charles Grant	56
2	Charles Grant	169
2	Charles Grant	209
2	Charles Grant	223
2	Charles Grant	230
2	Charles Grant	232

5. SOLUTION QUERY:

```
SELECT
    P.product_id,
    P.name,
    OI.quantity
FROM products P
LEFT JOIN order_items OI
ON P.product_id = OI.product_id;
```

OUTPUT:

Limit to 1000 rows

```

44 -- 5. List all products along with order item quantity (LEFT JOIN)
45 • SELECT
46     P.product_id,
47     P.name,
48     OI.quantity
49 FROM products P
50 LEFT JOIN order_items OI
51 ON P.product_id = OI.product_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	product_id	name	quantity
▶	1	Plant No	1
	1	Plant No	3
	1	Plant No	1
	1	Plant No	1
	1	Plant No	2
	1	Plant No	3
	1	Plant No	2
	1	Plant No	2
	1	Plant No	2
	1	Plant No	2
	1	Plant No	3
	1	Plant No	3
	1	Plant No	2
	1	Plant No	2
	1	Plant No	2
	1	Plant No	3
	1	Plant No	1

Result 3 x

6. SOLUTION QUERY:

```

SELECT
    O.order_id,
    PT.payment_id,
    PT.method,
    PT.amount_paid
FROM orders O
RIGHT JOIN payments PT
ON O.order_id = PT.order_id;

```

OUTPUT:

Limit to 1000 rows

```

53 -- 6. List all payments including those with no matching orders (RIGHT JOIN)
54 • SELECT
55     O.order_id,
56     PT.payment_id,
57     PT.method,
58     PT.amount_paid
59 FROM orders O
60 RIGHT JOIN payments PT
61 ON O.order_id = PT.order_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

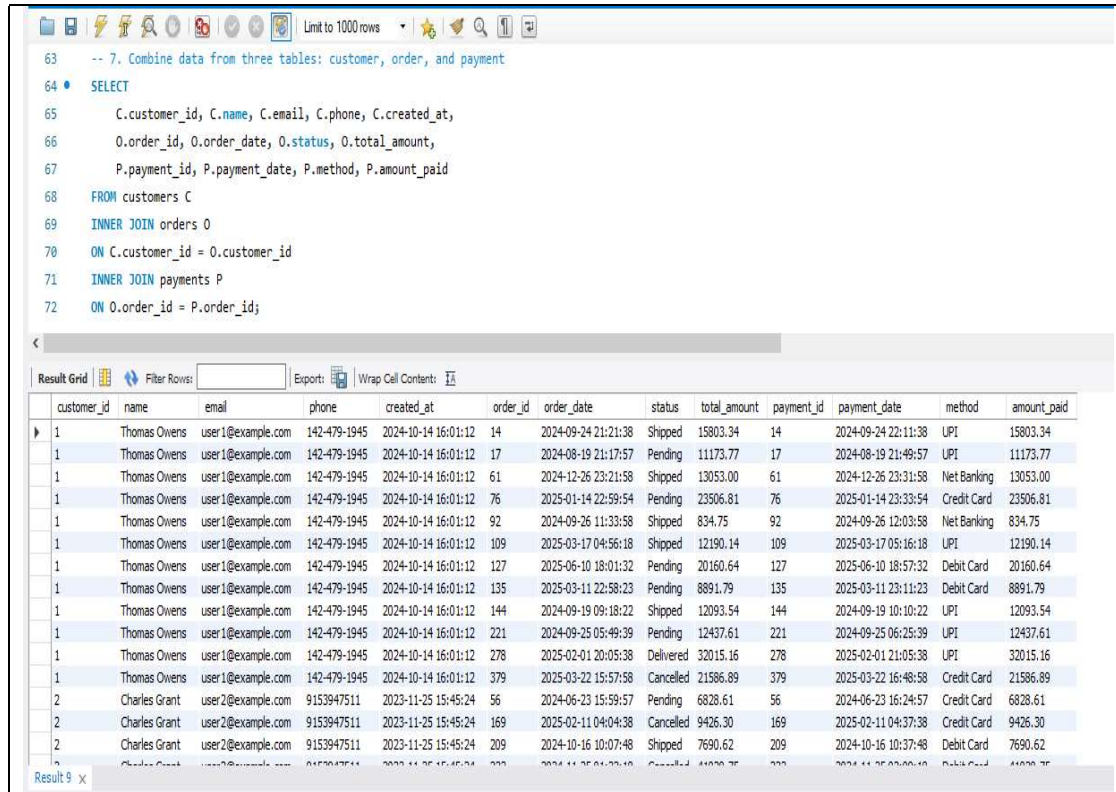
	order_id	payment_id	method	amount_paid
▶	1	1	Credit Card	9414.28
	2	2	Net Banking	532.20
	3	3	Credit Card	5164.56
	4	4	UPI	9469.78
	5	5	UPI	14501.86
	6	6	UPI	31050.17
	7	7	UPI	3043.67
	8	8	Net Banking	32714.06
	9	9	Net Banking	24219.20
	10	10	Debit Card	24342.52
	11	11	Credit Card	16196.13
	12	12	Credit Card	9890.72
	13	13	UPI	9627.36
	14	14	UPI	15803.34
	15	15	UPI	12421.88
	16	16	UPI	22856.73
	17	17	UPI	11173.77

Result 4 x

7. SOLUTION QUERY:

```
SELECT
    C.customer_id, C.name, C.email, C.phone, C.created_at,
    O.order_id, O.order_date, O.status, O.total_amount,
    P.payment_id, P.payment_date, P.method, P.amount_paid
FROM customers C
INNER JOIN orders O
ON C.customer_id = O.customer_id
INNER JOIN payments P
ON O.order_id = P.order_id;
```

OUTPUT:



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
-- 7. Combine data from three tables: customer, order, and payment
SELECT
    C.customer_id, C.name, C.email, C.phone, C.created_at,
    O.order_id, O.order_date, O.status, O.total_amount,
    P.payment_id, P.payment_date, P.method, P.amount_paid
FROM customers C
INNER JOIN orders O
ON C.customer_id = O.customer_id
INNER JOIN payments P
ON O.order_id = P.order_id;
```

Below the query, the 'Result Grid' shows the output. The table has 12 columns: customer_id, name, email, phone, created_at, order_id, order_date, status, total_amount, payment_id, payment_date, method, and amount_paid. The results are as follows:

customer_id	name	email	phone	created_at	order_id	order_date	status	total_amount	payment_id	payment_date	method	amount_paid
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	2024-09-24 21:21:38	Shipped	15803.34	14	2024-09-24 22:11:38	UPI	15803.34
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	2024-08-19 21:17:57	Pending	11173.77	17	2024-08-19 21:49:57	UPI	11173.77
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	2024-12-26 23:21:58	Shipped	13053.00	61	2024-12-26 23:31:58	Net Banking	13053.00
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	2025-01-14 22:59:54	Pending	23506.81	76	2025-01-14 23:33:54	Credit Card	23506.81
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	2024-09-26 11:33:58	Shipped	834.75	92	2024-09-26 12:03:58	Net Banking	834.75
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	2025-03-17 04:56:18	Shipped	12190.14	109	2025-03-17 05:16:18	UPI	12190.14
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	2025-06-10 18:01:32	Pending	20160.64	127	2025-06-10 18:57:32	Debit Card	20160.64
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	2025-03-11 22:58:23	Pending	8891.79	135	2025-03-11 23:11:23	Debit Card	8891.79
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	2024-09-19 09:18:22	Shipped	12093.54	144	2024-09-19 10:10:22	UPI	12093.54
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	2024-09-25 05:49:39	Pending	12437.61	221	2024-09-25 06:25:39	UPI	12437.61
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	2025-02-01 20:05:38	Delivered	32015.16	278	2025-02-01 21:05:38	UPI	32015.16
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	2025-03-22 15:57:58	Cancelled	21586.89	379	2025-03-22 16:48:58	Credit Card	21586.89
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	56	2024-06-23 15:59:57	Pending	6828.61	56	2024-06-23 16:24:57	Credit Card	6828.61
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	169	2025-02-11 04:04:38	Cancelled	9426.30	169	2025-02-11 04:37:38	Credit Card	9426.30
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	209	2024-10-16 10:07:48	Shipped	7690.62	209	2024-10-16 10:37:48	Debit Card	7690.62

LEVEL 5: SUBQUERIES (INNER QUERIES)

1. SOLUTION QUERY:

```
SELECT *
FROM products
WHERE price >
(
    SELECT AVG(price)
    FROM products
);
```

OUTPUT:

Limit to 1000 rows

```

5  -- 1. List all products priced above the average product price
6  • SELECT *
7    FROM products
8    WHERE price >
9      (
10     SELECT AVG(price)
11     FROM products
12  );

```

Result Grid

	product_id	name	category	price	stock_quantity	added_on
2		Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4		Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5		Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7		Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8		Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10		Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12		Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
18		Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19		Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20		Real Source	Books	4398.66	197	2025-02-08 10:28:27
21		Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22		Move Small	Books	4168.08	22	2023-11-12 18:26:30
24		Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25		South Free	Clothing	3543.58	44	2023-07-25 04:13:43
26		Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
29		Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
30		Future Sort	Home	3043.67	100	2025-05-17 06:55:28
33		Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
35		Fire Officer	Electronics	4734.80	108	2023-10-23 04:15:51

products 1 x

2. SOLUTION QUERY:

```

SELECT *
FROM customers
WHERE customer_id IN
(
    SELECT DISTINCT customer_id
    FROM orders
);

```

OUTPUT:

Limit to 1000 rows

```

14  -- 2. Find customers who have placed at least one order
15  • SELECT *
16    FROM customers
17    WHERE customer_id IN
18      (
19     SELECT DISTINCT customer_id
20     FROM orders
21  );

```

Result Grid

	customer_id	name	email	phone	created_at
1		Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2		Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3		Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4		Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5		David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6		Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7		Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8		John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9		Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10		Shawin Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11		Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
12		Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
13		Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14		Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
15		Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16		Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
17		Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
18		Jeffrey Bray	user18@example.com	164.962.0222x92	2024-07-02 19:51:55
19		Amenda Bright	user19@example.com	280.081.0708x60	2024-12-20 22:58:15

customers 8 x

3. SOLUTION QUERY:

```
SELECT *
FROM orders O
WHERE total_amount >
(
    SELECT AVG(total_amount)
    FROM orders
    WHERE customer_id = O.customer_id
);
```

OUTPUT:

<

4. SOLUTION QUERY:

```
SELECT *
FROM customers
WHERE customer_id NOT IN
(
    SELECT DISTINCT customer_id
    FROM orders
);
```

OUTPUT:

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
33
34 -- 4. Display customers who haven't placed any orders
35 • SELECT *
36   FROM customers
37  WHERE customer_id NOT IN
38  (
39      SELECT DISTINCT customer_id
40      FROM orders
41  );
```

Below the query editor is a "Result Grid" section. It includes a "Filter Rows:" input field and buttons for "Edit:", "Export/Import:", and "Wrap Cell Content:". The table structure is defined with columns: customer_id, name, email, phone, and created_at. The table is currently empty, with a single row header visible.

At the bottom of the IDE, a tab labeled "customers 7" is visible.

5. SOLUTION QUERY:

```
SELECT *
FROM products
WHERE product_id NOT IN
(
    SELECT DISTINCT product_id
    FROM order_items
);
```

OUTPUT:

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
43 -- 5. Show products that were never ordered
44 • SELECT *
45   FROM products
46  WHERE product_id NOT IN
47  (
48      SELECT DISTINCT product_id
49      FROM order_items
50  );
```

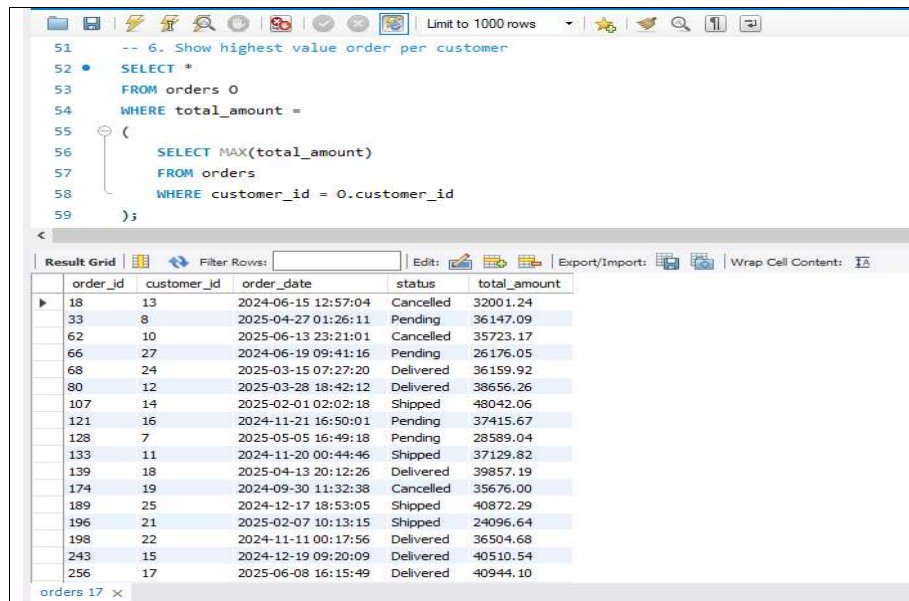
Below the query editor is a "Result Grid" section. It includes a "Filter Rows:" input field and buttons for "Edit:", "Export/Import:", and "Wrap Cell Content:". The table structure is defined with columns: product_id, name, category, price, stock_quantity, and added_on. The table is currently empty, with a single row header visible.

At the bottom of the IDE, a tab labeled "products 1" is visible.

6. SOLUTION QUERY:

```
SELECT *
FROM orders O
WHERE total_amount =
(
    SELECT MAX(total_amount)
    FROM orders
    WHERE customer_id = O.customer_id
);
```

OUTPUT:



The screenshot shows a database query editor with a SQL query and its results. The query is designed to find the highest total amount for each customer. The results are displayed in a grid with columns: order_id, customer_id, order_date, status, and total_amount. The results are sorted by total_amount in descending order.

order_id	customer_id	order_date	status	total_amount
18	13	2024-06-15 12:57:04	Cancelled	32001.24
33	8	2025-04-27 01:26:11	Pending	36147.09
62	10	2025-06-13 23:21:01	Cancelled	35723.17
66	27	2024-06-19 09:41:16	Pending	26176.05
68	24	2025-03-15 07:27:20	Delivered	36159.92
80	12	2025-03-28 18:42:12	Delivered	38656.26
107	14	2025-02-01 02:02:18	Shipped	48042.06
121	16	2024-11-21 16:50:01	Pending	37415.67
128	7	2025-05-05 16:49:18	Pending	28589.04
133	11	2024-11-20 00:44:46	Shipped	37129.82
139	18	2025-04-13 20:12:26	Delivered	39857.19
174	19	2024-09-30 11:32:38	Cancelled	35676.00
189	25	2024-12-17 18:53:05	Shipped	40872.29
196	21	2025-02-07 10:13:15	Shipped	24096.64
198	22	2024-11-11 00:17:56	Delivered	36504.68
243	15	2024-12-19 09:20:09	Delivered	40510.54
256	17	2025-06-08 16:15:49	Delivered	40944.10

7. SOLUTION QUERY:

```
SELECT
    C.customer_id, C.name,
    O.order_id, O.total_amount
FROM customers C
INNER JOIN orders O
ON C.customer_id = O.customer_id
WHERE O.total_amount =
(SELECT MAX(total_amount)
 FROM orders
 WHERE customer_id = O.customer_id);
```

OUTPUT:

The screenshot shows a SQL IDE window with a query editor and a result grid. The query is titled "7. Highest Order Per Customer (Including Names)". The result grid displays 15 rows of data, sorted by total_amount in descending order.

```

62 -- 7. Highest Order Per Customer (Including Names)
63 • SELECT
64     C.customer_id, C.name,
65     O.order_id, O.total_amount
66 FROM customers C
67 INNER JOIN orders O
68 ON C.customer_id = O.customer_id
69 WHERE O.total_amount =
70     (SELECT MAX(total_amount)
71      FROM orders
72      WHERE customer_id = O.customer_id);

```

	customer_id	name	order_id	total_amount
1	1	Thomas Owens	278	32015.16
2	2	Charles Grant	315	42056.04
3	3	Kaitlin Richards	281	41679.11
4	4	Christina Williams	386	25747.34
5	5	David Allen	266	39921.78
6	6	Mark Duke	330	39003.19
7	7	Briana Wright	128	28589.04
8	8	John Bryan	33	36147.09
9	9	Jason Thompson	348	21414.44
10	10	Shawn Hill	62	35723.17
11	11	Walter Jenkins	133	37129.82
12	12	Mary Knight	80	38656.26
13	13	Leslie Wilson	18	32001.24
14	14	Deborah Arias	107	48042.06
15	15	Austin Flores	243	40510.54

LEVEL 6: SET OPERATIONS

1. SOLUTION QUERY:

```

SELECT *
FROM customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM orders
    UNION
    SELECT customer_id
    FROM product_reviews
);

```

OUTPUT:

```

5  -- 1. List all customers who have either placed an order or written a product review
6  • SELECT *
7    FROM customers
8    WHERE customer_id IN
9      (
10     SELECT customer_id
11     FROM orders
12     UNION
13     SELECT customer_id
14     FROM product_reviews
15   );

```

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35

2. SOLUTION QUERY:

VERSION 1: INTERSECT NOT SUPPORTED

SELECT DISTINCT

C.*

FROM customers C

INNER JOIN orders O

ON C.customer_id = O.customer_id

INNER JOIN product_reviews PR

ON C.customer_id = PR.customer_id;

OUTPUT:

```

19  -- Version 1: INTERSECT not supported
20  • SELECT DISTINCT
21    C.*
22  FROM customers C
23  INNER JOIN orders O
24  ON C.customer_id = O.customer_id
25  INNER JOIN product_reviews PR
26  ON C.customer_id = PR.customer_id;

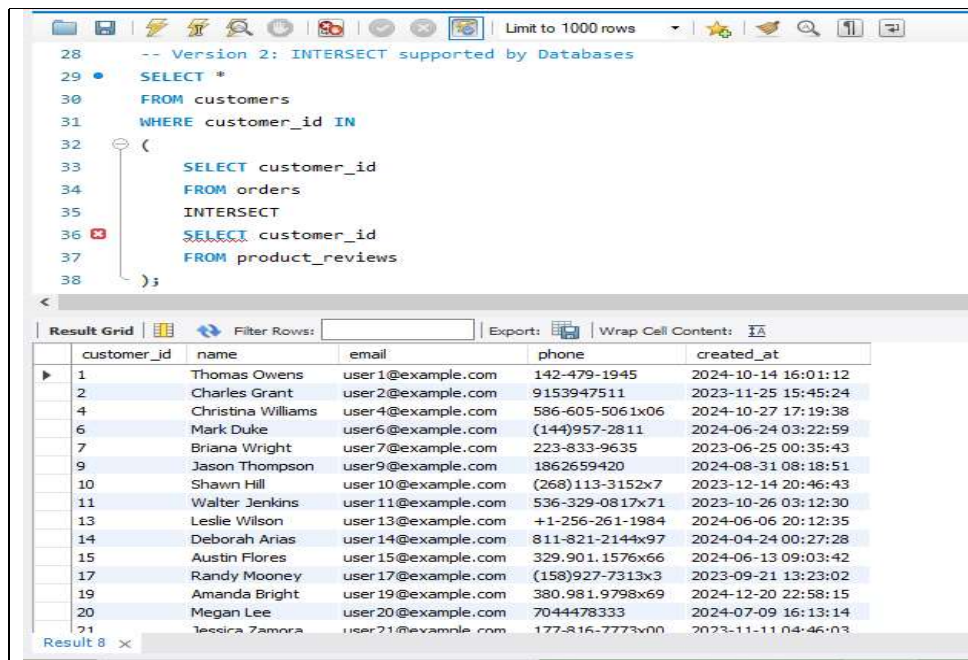
```

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14
21	Jessica Zamora	user21@example.com	177-816-7773x00	2023-11-11 04:46:03
22	Peter Phillips	user22@example.com	389-656-1695	2025-05-22 06:12:06
23	Kathryn Mathews	user23@example.com	001-707-722-178	2023-12-21 15:30:24
25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54

VERSION 2: INTERSECT SUPPORTED BY DATABASES

```
SELECT *
FROM customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM orders
    INTERSECT
    SELECT customer_id
    FROM product_reviews
);
```

OUTPUT:



The screenshot shows a database query editor with a toolbar at the top. The SQL query is entered in the main text area. Below the query, a 'Result Grid' is displayed, showing a table with 5 columns: customer_id, name, email, phone, and created_at. The table contains 21 rows of data, representing customers who have both placed orders and written product reviews.

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14
21	Jessica Zamora	user21@example.com	177-816-7773x00	2023-11-11 04:46:03